Languages for AI
June 26 2020

Time: 2 hours.

In the following $A, B, \ldots$ are propositional variables, $a, b, \ldots$ constant symbols, $f, g, \ldots$ function symbols, $X, Y \ldots$ variables, $p, q, \ldots$ predicate symbols and $\phi, \psi, \ldots$ formulas (unless differently specified).

1. (4 points) Consider the language of propositional logic. Use natural deduction to prove that the following holds, or find a counter-example to show that it does not hold

   - $\vdash \phi \to (\psi \to (\sigma \to (\phi \wedge \psi \wedge \sigma)))$
   - $\vdash \phi \to ((\neg\phi \to \psi) \to \psi)$

   Hint: Assume that you can use the rule for $\vee$ introduction

   $$\frac{\phi}{\phi \vee \psi}$$

2. (4 points) Transform the following propositional logic formula into an equivalent formula in Disjunctive Normal Form

   - $\neg A \wedge (\neg B \wedge ((C \wedge D) \to A))$

3. (4 points) Prove that that $\phi \models \psi$ (ψ is a logical consequence of ϕ) or that $\phi \not\models \psi$ for the following formulas:

   - $\phi : \neg A \vee \neg B$ and $\psi : (A \wedge \neg B) \to (B \vee C)$
   - $\phi : A \to C$ and $\psi : (A \to B) \to C$

4. Define a propositional language which allows to describe the state of an (simplified) ATM at different instants. With the language defined above provide a (set of) formulas which expresses the following facts:

   (a) At each instant the ATM is one (and only one) of these fours states: idle, checking the card, delivering the money, error;

   (b) Under normal functioning the ATM switches from idle to checking the card, from checking the card to delivering the money and from to delivering the money to idle;

   (c) When the machine enter in an errore state it remains in this state for all the successive instants.

5. (5 points) Consider the following definitions for a graph G:

   - Adjacent node: a node x is adjacent to node y iff there exists and edge between x and y.
   - Path: a path in a graph is a sequence of edges which joins a sequence of nodes.
   - Connected component: A connected component in G is a subgraph of G in which any two vertices are connected to each other by paths, and which is connected to no additional vertices in the supergraph G.
   - Dummy graph coloring problem: given a non-oriented graph, associate a color to each of its nodes in such a way that adjacent nodes have the same color.

   Provide a FOL language and a set of axioms that formalize the dummy graph coloring problem of a graph with k connected components, maximizing the number of colors used.

6. (5 points) A binary tree is either empty or it is composed of a root element and two successors, which are binary trees themselves. In Prolog we represent the empty tree by the atom `nil` and the non-empty tree by the term `t(X,L,R)` where X denotes the root node and L and R denote the left and right subtree, respectively. A leaf is a node with no successors. For example, the tree in the Figure is represented by the term `t(a,t(b,nil,nil),t(c,nil,nil))` and b and c are leaves.

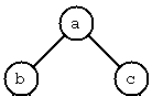

Figure 1: A tree

   Write a Prolog program which defines a predicate `count_leaves_parents(T,N)` which counts the number of nodes which have only leafs as chlidren have

```
% count_lnot _eaves(T,N) :- the binary tree T has N nodes which
% have only leafs as children.
```

7. (4 points) Provide a Prolog program P and a goal G such that the evaluation of G in P successfully terminates, while the evaluation of G in a program P', obtained from P by changing the order of clauses, does not terminate.

8. (3 points) Describe shortly the difference between symbolic computation and sub-symbolic computation in the context of AI.

Es. 1

1) $\Phi \rightarrow [\Psi \rightarrow (\sigma \rightarrow (\Phi \wedge \Psi \wedge \sigma))] \overset{?}{\equiv} \bot$

$\Phi \equiv \top$

$[\Psi \rightarrow (\sigma \rightarrow (\top \wedge \Psi \wedge \sigma))] \equiv \bot$

$\Psi \equiv \top$

$(\sigma \rightarrow (\top \wedge \top \wedge \sigma)) \equiv \bot$

$\sigma \equiv \top$

$(\top \wedge \top \wedge \sigma \equiv \bot \quad \sigma \equiv \bot$ contraddiction so the original is always true

2) $\Phi \rightarrow [(\neg \Phi \rightarrow \Psi) \rightarrow \Psi] \overset{?}{\equiv} \bot$

$\Phi \equiv \top$

$[(\bot \rightarrow \Psi) \rightarrow \Psi] \equiv \bot$

$\bot \rightarrow \Psi \equiv \top \quad \Psi \equiv \bot$ so the original proposition can be false when $\Phi \equiv \top$ and $\Psi \equiv \bot$

$\Psi \equiv \bot$

Es. 2

$\neg A \wedge (\neg B \wedge ((C \wedge D) \rightarrow A)) \overset{DNF}{\equiv} \neg A \wedge (\neg B \wedge [\neg (C \wedge D) \vee A]) \equiv \neg A \wedge \neg B \wedge [(\neg C \vee \neg D \vee A)] \equiv$

$\equiv (\neg A \wedge \neg B \wedge \neg C) \vee (\neg A \wedge \neg B \wedge \neg D) \vee (\neg A \wedge \underbrace{\neg B \wedge A}_{\bot}) \equiv (\neg A \wedge \neg B \wedge \neg C) \vee (\neg A \wedge \neg B \wedge \neg D)$

Es. 3

1) $[\neg A \vee \neg B] \longrightarrow [(A \wedge \neg B) \rightarrow B \vee C] \overset{?}{\equiv} \bot$ it is not a logical consequence when $A \equiv \top, B \equiv \bot, C \equiv \bot$

$[\neg A \vee \neg B] \equiv \top$

$[(A \wedge \neg B) \rightarrow B \vee C] \equiv \bot$

$A \wedge \neg B \equiv \top \quad A \equiv \top, B \equiv \bot$

$\bot \vee C \equiv \bot \quad C \equiv \bot$

2) $[A \rightarrow C] \longrightarrow [(A \rightarrow B) \rightarrow C] \overset{?}{\equiv} \bot$ so it is not a logical consequence when A, B, C are all false.

$A \rightarrow C \equiv \top$ so $A \rightarrow \bot \equiv \top$ iff $A \equiv \bot$

$(A \rightarrow B) \rightarrow C \equiv \bot$

$A \rightarrow B \equiv \top$ so $\bot \rightarrow B \equiv \top$ iff $B \equiv \bot$

$C \equiv \bot$

Es. 4

states: I_K, C_K, D_K, E_K in K instant

a) $[I_K \leftrightarrow (\neg C_K \wedge \neg D_K \wedge \neg E_K)] \wedge [C_K \leftrightarrow (\neg I_K \wedge \neg D_K \wedge \neg E_K)] \wedge [D_K \leftrightarrow (\neg I_K \wedge \neg C_K \wedge \neg E_K)] \wedge [E_K \leftrightarrow (\neg I_K \wedge \neg C_K \wedge \neg D_K)]$

b) $[I_K \longrightarrow (I_{K+1} \vee C_{K+1} \vee E_{K+1})] \wedge [C_K \longrightarrow (C_{K+1} \vee D_{K+1} \vee E_{K+1})] \wedge [D_K \longrightarrow (D_{K+1} \vee I_{K+1} \vee E_{K+1})]$

IT USES THE XOR CONDITION OF POINT (a) IMPLICITLY

c) $E_K \longrightarrow E_{K+1}$

Es. 5

$adj(x, y)$, $color(x)$, $path(x, y)$

1) $\forall x_1, x_2$: $adj(x_1, x_2) \longrightarrow [color(x_1) = color(x_2)]$

2) $\forall x_1, x_2$: $path(x_1, x_2) \longleftrightarrow [\exists z: adj(x_1, z) \wedge path(z, x_2)] \vee adj(x_1, x_2)$

3) $\forall x_1, x_2$: $\neg path(x_1, x_2) \longrightarrow [color(x_1) \neq color(x_2)]$

Es. 6

```
count_leaves_parent(t(_, [], t(_,[],[])), 1) :-!.

count_leaves_parent(t(_, t(_,[],[]), []), 1) :-!.

count_leaves_parent(t(_, t(_,[],[]), t(_,[],[])), 1) :-!.

count_leaves_parent(t(_, L, R), N) :-
    count_leaves_parent(L, N1),
    count_leaves_parent(R, N2),
    N is N1 + N2.
```